

Natural Language processing

Section 5

implementation of RNN on Arabic Dataset

Dataset

- we'll be using the **Arabic Sentiment Twitter Corpus** from Kaggle.
- This dataset was collected to provide Arabic sentiment corpus for the research community to investigate deep learning approaches for Arabic sentiment analysis.
- This dataset we collected in April 2019. It contains 58K Arabic tweets (47K training, 11K test) tweets annotated in positive and negative labels. The dataset is balanced and collected using positive and negative emojis lexicon.

<https://www.kaggle.com/datasets/mksaad/arabic-sentiment-twitter-corpus>

 neg	10/31/2025 2:40 PM	File folder	
 pos	10/31/2025 2:41 PM	File folder	
 test_Arabic_tweets_negative.tsv	7/27/2020 9:01 PM	TSV File	681 KB
 test_Arabic_tweets_positive.tsv	7/27/2020 9:01 PM	TSV File	758 KB
 train_Arabic_tweets_negative.tsv	7/27/2020 9:01 PM	TSV File	2,712 KB
 train_Arabic_tweets_positive.tsv	7/27/2020 9:01 PM	TSV File	3,048 KB

1-Import necessary libraries

[]

```
import pandas as pd
import numpy as np
import re
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense, Dropout
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
```

2- Read the data using pandas then combine (pos_train , neg_train) for the training
And (pos_test , neg_test) for testing

```
[ ] train_pos = "train_Arabic_tweets_positive_train.tsv"
    train_neg = "train_Arabic_tweets_negative_train.tsv"
    test_pos  = "test_Arabic_tweets_positive_test.tsv"
    test_neg  = "test_Arabic_tweets_negative_test.tsv"

[ ] train_pos_df = pd.read_csv(train_pos, sep='\t', names=['label', 'text'], quoting=3)
    train_neg_df = pd.read_csv(train_neg, sep='\t', names=['label', 'text'], quoting=3)
    test_pos_df  = pd.read_csv(test_pos,  sep='\t', names=['label', 'text'], quoting=3)
    test_neg_df  = pd.read_csv(test_neg,  sep='\t', names=['label', 'text'], quoting=3)

    train_df = pd.concat([train_pos_df, train_neg_df], ignore_index=True)
    test_df  = pd.concat([test_pos_df, test_neg_df], ignore_index=True)
```

3- Updated Data info

```
# Display info
len(train_df)
```

⇒ 47000

```
[ ] len(test_df)
```

11751

4- Check class distribution for the training data



5- Preprocessing the data

```
[ ]  
  
def clean_arabic(text):  
    if isinstance(text, str):  
        text = re.sub(r'^\u0600-\u06FF\s', '', text) # Keep Arabic only  
        text = re.sub(r'\s+', ' ', text).strip()  
        return text  
    return ""  
  
train_df['clean_text'] = train_df['text'].apply(clean_arabic)  
test_df['clean_text'] = test_df['text'].apply(clean_arabic)  
  
# Check before/after cleaning  
pd.DataFrame({  
    'Before': train_df['text'].head(5),  
    'After': train_df['clean_text'].head(5)  
})
```

	Before	After
0	...نحن الذين يتحول كل ما نود أن نقوله إلى دعاء لل	...نحن الذين يتحول كل ما نود أن نقوله إلى دعاء لل
1	...وفي النهاية لن يبقى معك أحداً إلا من رأى الجمال	...وفي النهاية لن يبقى معك أحداً إلا من رأى الجمال
2	من الخير نفسه 🧡	من الخير نفسه
3	...زائل الملعب يصردنا يلعب كن عالي الهمة ولا ترض #	...زائل الملعب يصردنا يلعب كن عالي الهمة ولا ترضى ين
4	...الشيء الوحيد الذي وصلوا فيه للعالمية هو : المس	...الشيء الوحيد الذي وصلوا فيه للعالمية هو المصيا

6- Encode the 2 labels then tokenize and pad the x_train , x_test

```
[ ]  
# Encode 'pos'/'neg' → 1/0  
le = LabelEncoder()  
train_df['label_enc'] = le.fit_transform(train_df['label'])  
test_df['label_enc'] = le.transform(test_df['label'])  
  
tokenizer = Tokenizer(num_words=20000, oov_token="<OOV>")  
tokenizer.fit_on_texts(train_df['clean_text'])  
  
max_len = 40 # Max sequence length  
X_train = pad_sequences(tokenizer.texts_to_sequences(train_df['clean_text']), maxlen=max_len, padding='post')  
X_test = pad_sequences(tokenizer.texts_to_sequences(test_df['clean_text']), maxlen=max_len, padding='post')  
  
y_train = np.array(train_df['label_enc'])  
y_test = np.array(test_df['label_enc'])  
  
len(tokenizer.word_index)
```

70970

7- Build the RNN model

```
[ ]  
vocab_size = len(tokenizer.word_index) + 1  
embedding_dim = 100  
  
model = Sequential([  
    Embedding(vocab_size, embedding_dim, input_length=max_len),  
    SimpleRNN(128, activation='tanh'),  
    Dropout(0.3),  
    Dense(64, activation='relu'),  
    Dense(1, activation='sigmoid')  
)  
  
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])  
model.summary()
```

8-Train the model

```
[ ]
```

```
history = model.fit(  
    X_train, y_train,  
    epochs=5,  
    batch_size=128,  
    validation_data=(X_test, y_test),  
    verbose=1  
)
```

Epoch 1/5

368/368 ————— **51s** 128ms/step - accuracy: 0.6530 - loss: 0.5827 - val_accuracy: 0.7515 - val_loss: 0.4624

Epoch 2/5

368/368 ————— **82s** 129ms/step - accuracy: 0.8277 - loss: 0.3608 - val_accuracy: 0.7639 - val_loss: 0.4688

Epoch 3/5

368/368 ————— **47s** 129ms/step - accuracy: 0.9031 - loss: 0.2263 - val_accuracy: 0.7570 - val_loss: 0.5690

Epoch 4/5

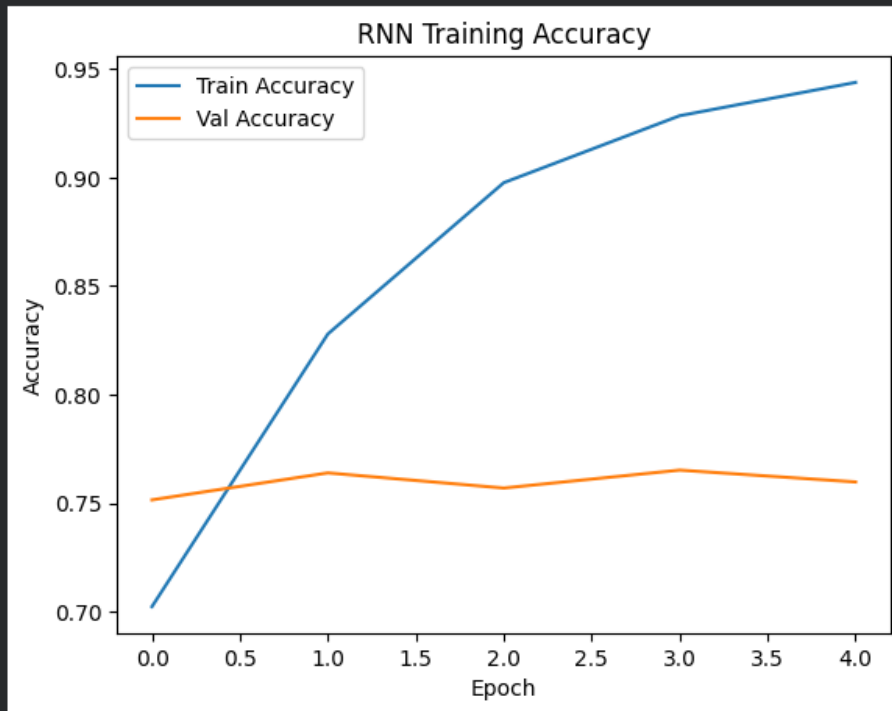
368/368 ————— **46s** 126ms/step - accuracy: 0.9352 - loss: 0.1562 - val_accuracy: 0.7652 - val_loss: 0.5888

Epoch 5/5

368/368 ————— **48s** 130ms/step - accuracy: 0.9476 - loss: 0.1289 - val_accuracy: 0.7598 - val_loss: 0.6683

9- Evaluation

```
[ ] plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('RNN Training Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```



10-Test accuracy and predicting

[]



```
loss, acc = model.evaluate(X_test, y_test)
print(f"\n Test Accuracy: {acc*100:.2f}%")
```



368/368 ————— 2s 6ms/step - accuracy: 0.7455 - loss: 0.6844

Test Accuracy: 75.98%

[]

```
def predict_sentiment(text):
    seq = tokenizer.texts_to_sequences([clean_arabic(text)])
    pad = pad_sequences(seq, maxlen=max_len, padding='post')
    pred = model.predict(pad)[0][0]
    return "Positive" if pred > 0.5 else "Negative"

print(predict_sentiment("برنامج وحش"))
```

1/1 ————— 0s 50ms/step
Negative